

torch.autograd.Function.backward#

[https://pytorch.org/docs/stable/generated/torch.autograd.Function.backward.h](https://pytorch.org/docs/stable/generated/torch.autograd.Function.backward.html)

Description:

Define a formula for differentiating the operation with backward mode automatic differentiation. This function is to be overridden by all subclasses. (Defining this function is equivalent to defining the vjp function.) It must accept a context `ctx` as the first argument, followed by as many outputs as the `forward()` returned (None will be passed in for non tensor outputs of the forward function), and it should return as many tensors, as there were inputs to `forward()`. Each argument is the gradient w.r.t the given output, and each returned value should be the gradient w.r.t. the corresponding input. If an input is not a Tensor or is a Tensor not requiring grads, you can just pass None as a gradient for that input. The context can be used to retrieve tensors saved during the forward pass. It also has an attribute `ctx.needs_input_grad` as a tuple of booleans representing whether each input needs gradient. E.g., `backward()` will have `ctx.needs_input_grad[0] = True` if the first input to `forward()` needs gradient computed w.r.t. the output.

Function Signature

```
static Function.backward(ctx, *grad_outputs)[source]#
```

Return type

Returns:

Any

Detailed Documentation

Documentation

Define a formula for differentiating the operation with backward mode automatic differentiation.

This function is to be overridden by all subclasses. (Defining this function is equivalent to defining the vjp function.)

It must accept a context `ctx` as the first argument, followed by as many outputs as the `forward()` returned (None will be passed in for non tensor outputs of the forward function), and it should return as many tensors, as there were inputs to `forward()`. Each argument is the gradient w.r.t the given output, and each returned value should be the gradient w.r.t. the corresponding input. If an input is not a Tensor or is a Tensor not requiring grads, you can just pass None as a gradient for that input.

The context can be used to retrieve tensors saved during the forward pass. It also has an attribute `ctx.needs_input_grad` as a tuple of booleans representing whether each input needs gradient. E.g., `backward()` will have `ctx.needs_input_grad[0] = True` if the first input to `forward()` needs gradient computed w.r.t. the output.